SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr G.A. SENTHIL

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

ANSWERS:

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain.

Solution

Problem Statement

Design an RL agent that learns to navigate a Grid World environment while avoiding obstacles and reaching the goal in the shortest path.

Constraints

- The agent cannot move outside the grid.
- Obstacles cannot be crossed.
- The goal must be reached in minimum steps.
- Each movement consumes one step.

RL Components

Component	Description
Agent	Robot
Environment	Grid World
States	Each grid cell
Actions	Up, Down, Left, Right
Reward	+100 Goal, -10 Obstacle, -1 Step
Policy	Shortest safe path

Python Program (Q-Learning)

Python

Run

```
import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=False)

q = np.zeros((env.observation_space.n,
              env.action_space.n))

alpha = 0.8
gamma = 0.95
epsilon = 0.1

for episode in range(500):

    state, info = env.reset()
```

```
    done = False

    while not done:

        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(q[state])

        next_state, reward, terminated, truncated, info = env.step(action)

        q[state, action] += alpha * (
            reward +
            gamma * np.max(q[next_state])
            - q[state, action]
        )

        state = next_state
        done = terminated or truncated

print(q)
```

Explanation

The agent explores the Grid World using the ε-greedy strategy. Over multiple episodes, the Q-table is updated using the Bellman equation. Eventually, the agent learns the safest and shortest path to the goal while avoiding obstacles.

2. Implement an ε-Greedy Multi-Armed Bandit algorithm under a budget of 500 trials. Compare ε = 0.1, 0.3, and 0.5.

Constraints

- Total trials = 500
- Limited exploration budget
- Maximize cumulative reward

Python Program

Python

Run

```
import numpy as np

true_rewards = [1, 3, 5]

def bandit(epsilon):

    values = np.zeros(3)
    counts = np.zeros(3)

    total = 0

    for i in range(500):

        if np.random.rand() < epsilon:
            arm = np.random.randint(3)
```

```
        else:
            arm = np.argmax(values)

        reward = true_rewards[arm]

        counts[arm] += 1

        values[arm] += (reward-values[arm])/counts[arm]

        total += reward

    print("Epsilon =", epsilon)
    print("Reward =", total)

bandit(0.1)
bandit(0.3)
bandit(0.5)
```

Comparison

ε Value	Exploration	Exploitation	Expected Reward
0.1	Low	High	Highest
0.3	Medium	Medium	Balanced
0.5	High	Low	Lower

Explanation

- ε = 0.1 performs more exploitation, usually giving the highest reward.
- ε = 0.3 balances exploration and exploitation.
- ε = 0.5 explores frequently, reducing cumulative reward within the fixed trial budget.

3. Design an MDP for an autonomous robot minimizing energy consumption.

Constraints

- Battery is limited.
- Robot must reach the destination.
- Avoid obstacles.
- Minimize energy consumption.

MDP Design

Component	Description
States	Robot location, battery level
Actions	Up, Down, Left, Right, Stop
Transition Probability	Probability of reaching the next state
Reward	+100 Goal, -1 Energy, -20 Collision

Component	Description
Discount Factor	0.9

Energy Constraint
Each movement consumes 1 unit of energy.
Maximum available energy:
Battery = 50 Units
Robot must reach the destination before energy becomes zero.
Reward Function
Goal = +100

Normal Move = -1

Obstacle = -20

Battery Empty = -100
Explanation
The optimal policy minimizes energy usage while safely reaching the goal. The robot avoids unnecessary movements because every step reduces the available battery.

4. Using Python and TensorFlow/Keras, implement Value Iteration for a constrained grid environment. Explain Bellman equations.
Constraints

- Restricted cells cannot be entered.
- Goal state provides maximum reward.
- Every movement has a cost.

Python Program
Python
Run

```
import numpy as np

gamma = 0.9

reward = np.array([[0,0,0],
                   [0,-1,0],
                   [0,0,100]])

value = np.zeros((3,3))

for k in range(20):

    new = value.copy()

    for i in range(3):
        for j in range(3):

            new[i,j]=reward[i,j]+gamma*value[i,j]
```

value=new

print(value)
Bellman Equation
where:

- = Value of current state
- = Immediate reward
- = Discount factor
- = Value of next state

Explanation
Value Iteration repeatedly updates state values using the Bellman optimality equation until the values converge. Restricted cells are excluded from the policy, ensuring the agent finds the optimal valid path.

5. Warehouse robot with only 30-minute battery
Problem
A warehouse robot has only 30 minutes of battery and must complete the maximum number of deliveries.
Constraints

- Battery capacity = 30 minutes
- Every movement consumes battery.
- Battery cannot become negative.
- Robot must return before battery expires.
- Obstacles must be avoided.

RL Framework

Component	Description
Agent	Warehouse Robot
Environment	Warehouse
States	Position, Battery Level
Actions	Move, Pick, Deliver, Recharge
Reward	Delivery, Safe Movement
Policy	Maximum deliveries before battery ends

Reward Design

Event	Reward
Successful Delivery	+50
Correct Pickup	+20

Event	Reward
Battery Saved	+10
Collision	-30
Battery Exhausted	-100
Delay	-10

Policy
The robot should:

1. Choose the nearest delivery.
2. Avoid obstacles.
3. Minimize travel distance.
4. Return for charging if the remaining battery is insufficient.

Explanation
The reward function encourages completing deliveries while conserving battery power. Penalizing battery exhaustion and long routes helps the agent learn energy-efficient behavior. Over time, the robot learns to prioritize nearby deliveries, avoid collisions, and recharge before the battery is depleted, maximizing the number of successful deliveries within the 30-minute constraint.

Code of Conduct:

*I **Kandukuri.Kavya-192425283**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Kandukuri.Kavya-192425283

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly

Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided